Smart Dustbin — Sample Data Pack

Structured datasets for building & integrating the voice agent

Project: ESP32 Smart Dustbin · Generated 2026-11-03 · Reference snapshot time 14:10:00 IST

This pack mirrors the exact fields served by the Smart Dustbin FastAPI backend (`/api/status`, `/api/history`, `/api/device`, `/api/heartbeat`) and the dashboard `types.ts` model. The single-device backend has been extended here to a **6-bin fleet** so the voice agent can answer fleet-level questions ("which bins are full?", "is any sensor down?"). Use Section 1–4 as the data the agent reads, and Section 5–7 as the conversational layer (intents, utterances, grounded dialogs) to train and wire it up.

1 · Live fleet status snapshot

One row per bin = one `/api/status` response. This is the primary table the voice agent queries for "how full" and "collect now" questions.

Bin ID	Location	Fill %	Distance (cm)	Status	Alert	Online	Last reading
BIN-A1	Cafeteria Entrance	92	4.8	FULL - COLLECT NOW	YES	yes	14:09:52
BIN-B2	Library Lobby	64	21.6	ALMOST FULL	no	yes	14:09:47
BIN-C3	Hostel Block C	38	37.2	AVAILABLE	no	yes	14:09:55
BIN-D4	Admin Gate	85	9.0	FULL - COLLECT NOW	YES	yes	14:09:50
BIN-E5	Sports Complex	12	52.8	AVAILABLE	no	yes	14:09:58
BIN-F6	Parking Lot	71	17.4	NO RECENT DATA	no	OFFLINE	13:52:11 (stale)

Thresholds: AVAILABLE < 50% ≤ ALMOST FULL < 80% ≤ FULL. Alert fires only after 2 consecutive ≥80% readings (FULL_CONFIRMATIONS). A reading older than 60 s → status "NO RECENT DATA", alert cleared.

2 · `GET /api/status` — JSON per bin

Exact shape returned by the backend (BinStatus + nested DeviceInfo). Two contrasting examples.

FULL + alert (BIN-A1)


```
{
  "distance": 4.8,
  "fill": 92,
  "status": "FULL - COLLECT NOW",
  "updated": "14:09:52",
  "alert": true,
  "ts": 1762154384,
  "age": 8.0,
  "location": "Cafeteria Entrance",
  "device": {
    "device_id": "BIN-A1",
    "ip": "192.168.1.21",
    "rssi": -58,
    "uptime_s": 414200,
    "fw": "1.4.2",
    "sensor_ok": true,
    "sensor_failures": 0,
    "echo_idle": 0,
    "last_seen_age": 8.0,
    "online": true
  }
}
```

offline + sensor fault (BIN-F6)


```
{
  "distance": 17.4,
  "fill": 71,
  "status": "NO RECENT DATA",
  "updated": "13:52:11",
  "alert": false,
  "ts": 1762153323,
  "age": 1069.0,
  "location": "Parking Lot",
  "device": {
    "device_id": "BIN-F6",
    "ip": "192.168.1.26",
    "rssi": -82,
    "uptime_s": 36120,
    "fw": "1.4.1",
    "sensor_ok": false,
    "sensor_failures": 7,
    "echo_idle": 1,
    "last_seen_age": 1069.0,
    "online": false
  }
}
```

3 · `GET /api/history` — reading time-series

Rolling buffer of `{ts, distance, fill}` points (backend keeps the last 300). Sample below = BIN-A1 filling up over ~30 min, crossing ALMOST FULL → FULL. Lets the agent answer “how fast is it filling” / “when did it get full”.

Time	Epoch ts	Distance (cm)	Fill %	Status
13:40:00	1793693400	49.2	18	AVAILABLE
13:43:00	1793693580	45.6	24	AVAILABLE
13:46:00	1793693760	41.4	31	AVAILABLE
13:49:00	1793693940	36.0	40	AVAILABLE
13:52:00	1793694120	30.6	49	AVAILABLE
13:55:00	1793694300	25.2	58	ALMOST FULL
13:58:00	1793694480	20.4	66	ALMOST FULL
14:01:00	1793694660	15.6	74	ALMOST FULL
14:04:00	1793694840	11.4	81	FULL - COLLECT NOW
14:07:00	1793695020	7.2	88	FULL - COLLECT NOW
14:10:00	1793695200	4.8	92	FULL - COLLECT NOW

JSON form: `[{"ts":1762155000,"distance":49.2,"fill":18}, ...]`

4 · `/api/device` & `/api/heartbeat` — device health

Heartbeat every 10 s; device marked offline after 25 s of silence. Feeds the agent’s “is the bin online / is the sensor working” answers.

Bin ID	Online	RSSI (dBm)	Uptime	FW	Sensor OK	Failures	Echo idle	Last seen
BIN-A1	yes	-58	4d 19h	1.4.2	ok	0	0	8 s ago
BIN-B2	yes	-63	3d 8h	1.4.2	ok	0	0	8 s ago
BIN-C3	yes	-71	5d 22h	1.4.2	ok	0	0	8 s ago
BIN-D4	yes	-55	1d 3h	1.4.2	ok	0	0	8 s ago
BIN-E5	yes	-68	8d 16h	1.4.2	ok	0	0	8 s ago
BIN-F6	OFFLINE	-82	10h 2m	1.4.1	FAULT	7	1	17m 49s ago

Heartbeat POST body: `{"device_id":"BIN-F6","ip":"192.168.1.26","rssi":-82,"uptime_s":36120,"fw":"1.4.1","sensor_ok":false,"sensor_failures":7,"echo_idle":1}`

Alert & event log

Timestamp	Bin ID	Event	Detail
2026-11-03 13:58:40	**BIN-A1**	FULL_ALERT	fill 81% (2nd confirmation) → collect now
2026-11-03 13:52:11	**BIN-F6**	SENSOR_FAULT	7 failed reads, echo idle=1, marked offline
2026-11-03 13:40:05	**BIN-D4**	FULL_ALERT	fill 85% → collect now
2026-11-03 11:20:33	**BIN-A1**	RESET	collected; state cleared via /api/reset
2026-11-03 09:15:02	**BIN-B2**	CROSSTALK_REJECT	reading 1.2 cm < 2.0 cm min, discarded

5 · Voice agent — intent catalogue

The vocabulary the agent must recognise, each mapped to the backend call that answers it. Entities: **bin_id** (BIN-A1…F6), **location** (cafeteria, library, hostel, admin gate, sports, parking), **status_value** (available / almost full / full).

Intent	What the user wants	Backend source	Key slots
`check_bin_fill`	Fill level of one bin	/api/status (filter bin_id)	bin_id | location
`list_full_bins`	All bins needing collection	/api/status (all, alert/fill≥80)	—
`nearest_available_bin`	An empty bin near a place	/api/status + location	location
`bin_status_summary`	Overview of whole fleet	/api/status (all)	—
`check_device_health`	Is a bin online / sensor ok	/api/device	bin_id
`fill_trend`	How fast a bin is filling	/api/history	bin_id | window
`last_collected`	When a bin was emptied	alert/event log (RESET)	bin_id
`acknowledge_reset`	Mark a bin collected	POST /api/reset	bin_id

6 · Training utterances (NLU samples)

Paraphrase sets per intent — drop straight into Dialogflow / Rasa / Alexa interaction model. {slots} marked in braces.

`check_bin_fill`

- How full is {the cafeteria bin}?
- What's the level of {BIN-A1}?
- Is the {library} bin full?
- Give me the fill percentage of {bin A1}.

`list_full_bins`

- Which bins are full?
- What needs collecting right now?
- Show me all bins over eighty percent.
- Any bins ready for pickup?

`nearest_available_bin`

- Where's an empty bin near the {sports complex}?
- Find me an available bin by the {hostel}.
- Which bin close to {parking} isn't full?

`bin_status_summary`

- Give me a status update.
- How are all the bins doing?
- Fleet summary please.

`check_device_health`

- Is {BIN-F6} online?
- Is the {parking} sensor working?
- Are any sensors down?

`fill_trend`

- How fast is {the cafeteria bin} filling?
- Is {BIN-A1} filling quickly?

- Show the trend for {library}.

`last_collected`

- When was {BIN-A1} last emptied?
- When did we collect the {cafeteria} bin?

`acknowledge_reset`

- Mark {BIN-A1} as collected.
- Reset the {cafeteria} bin, we emptied it.
- Clear the alert on {bin A1}.

7 · Grounded dialog samples

End-to-end turns showing the spoken reply the agent should produce from the Section 1–4 data. Use as golden responses / few-shot examples.

User: “How full is the cafeteria bin?”

Agent: The Cafeteria Entrance bin, BIN-A1, is 92% full and flagged for collection now. The last reading was 8 seconds ago.

intent: `check_bin_fill` | data: `status(BIN-A1) → fill 92, status FULL, alert true`

User: “Which bins need collecting?”

Agent: Two bins are full: BIN-A1 at the Cafeteria Entrance (92%) and BIN-D4 at the Admin Gate (85%). Both are flagged collect-now.

intent: `list_full_bins` | data: `status(all) where fill≥80 or alert`

User: “Where’s an empty bin near the sports complex?”

Agent: The Sports Complex bin, BIN-E5, is only 12% full and available right now.

intent: `nearest_available_bin` | data: `status(all) + location=sports, lowest fill`

User: “Are any sensors down?”

Agent: Yes — BIN-F6 at the Parking Lot is offline. Its ultrasonic sensor has reported 7 failed reads and hasn’t checked in for about 18 minutes.

intent: `check_device_health` | data: `device(all) where sensor_ok=false or online=false`

User: “How fast is the cafeteria bin filling?”

Agent: BIN-A1 rose from 18% to 92% over the last 30 minutes — roughly 2.5% per minute. At this rate it stays full until it’s emptied.

intent: `fill_trend` | data: `history(BIN-A1) slope`

User: “We just emptied the cafeteria bin, mark it collected.”

Agent: Done. I’ve reset BIN-A1 at the Cafeteria Entrance and cleared its full alert. It now reads 0% and is available.

intent: `acknowledge_reset` | data: `POST /api/reset (BIN-A1)`

Integration note

The current backend is single-device and in-memory; to serve the fleet above, add a `device_id` key to the `latest`/`history` stores (dict keyed by id) and let `/api/status` accept an optional `?bin_id=` filter. The voice agent then calls that endpoint, maps spoken location names to bin IDs with the entity table in Section 5, and speaks the templated reply from Section 7.